



Induction de Compétences pour Agents IA

Apprendre, stocker et réutiliser au-delà de la fenêtre de contexte

Cours CMU · Graham Neubig · Advanced Topics in AI Agents



Table des Matières

Structure du cours en 4 parties, 12 sections

1

Introduction et Motivation

Problématique, objectif et questions clés

Partie I

2

Trois Niveaux de Mémorisation

Contexte · Artefacts externes · Poids du modèle

Partie I

3

Types d'Expérience à Conserver

Épisode · Faits · Compétences · Intersection

Partie I

4

Compétences Rédigées par l'Humain : Standard Agent Skills

Standard Agent Skills, structure, divulgation progressive

Partie II

5

Évaluation des Compétences : SkillsBench

Benchmark 2M compétences, 400 tâches, 18 modèles

Partie II

6

Fondations : Mémoire Externe et Réflexion

MEMGPT, Mem0, Reflection, RAG trajectoires

Partie II

7

Induction Automatique de Compétences

Apprentissage en ligne, AWM (texte), ASI (code)

Partie II

8

Représentation : Texte vs Code

Comparatif détaillé, résultats, PolySkill (OOP)

Partie II

9

Cycle de Vie des Compétences

Opérations CRUD, SkillWeaver, boucle raffinage

Partie III

10

Apprendre des Échecs (Synapse / ReasoningBench)

Synapse, stratégies correctives, qualité juge

Partie III

11

Gestion de la Taille de la Mémoire

Distraction, retrieval, cache LRU, fusion

Partie III

12

Apprentissage par Renforcement pour l'Induction

RL bout-en-bout, récompense réutilisation, ACL 2024

Partie IV



POURQUOI ?

La mémoire et ses niveaux

1

Section 1

2

Section 2

3

Section 3



1

Introduction et Motivation

Problématique, objectif et questions clés

Problématique

Les agents effectuant des tâches répétitives (ex: navigation web, achats en ligne) reproduisent souvent les mêmes sous-tâches :

- Aller sur un site
- Trouver la barre de recherche
- Saisir une requête
- Cliquer sur "Rechercher"

Objectif

Extraire la structure partagée entre tâches pour la réutiliser, au lieu de tout réapprendre à chaque fois.

Questions Clés

Question	Enjeu
Comment représenter une compétence ?	Actions brutes, texte abstrait, ou code ?
Quelles parties d'une tâche conserver ?	Identification des sous-tâches réutilisables
Comment gérer le cycle de vie ?	Ajout, mise à jour, suppression, fusion

2

Trois Niveaux de Mémorisation

Contexte · Artefacts externes · Poids du modèle

Niveau	Description	Avantages	Inconvénients
Fenêtre de contexte	Historique observations/actions dans le contexte du LLM	Accès rapide, pas d'infrastructure	Limité en taille (même 1M tokens)
Artefacts externes	Base de données, système de fichiers, compétences stockées	Transférable entre modèles, inspectable par humains, persistant	Nécessite gestion (lecture/écriture via outils)
Mise à jour des poids	Fine-tuning, RL	Connaissances internalisées	Spécifique au modèle, coûteux, opaque

****Point clé** : Les artefacts externes (compétences) offrent le meilleur compromis : portabilité, inspectabilité, et interface humain-agent naturelle.**

3

Types d'Expérience à Conserver

Épisode · Faits · Compétences · Intersection

3.1 Épisode Complet

- Séquence brute : observations, actions, messages utilisateur
- Fidélité maximale mais coût token élevé
- Contient du bruit non généralisable

3.2 Faits (Memory Systems Classiques)

- Connaissances sur l'utilisateur/environnement (ex: "l'utilisateur habite à Paris")
- Utilisé par ChatGPT/Claude (système "Memory")
- Proche de la **compaction** : LLM génère un résumé textuel stocké en base

3.3 Compétences (Spécifique Agents)

- **Connaissance réutilisable sur comment agir** dans un environnement
- Exemple : "Comment rechercher un produit sur ce site"
- Abstraction des détails de la tâche spécifique (quoi chercher) pour garder la méthode (comment chercher)

Intersection Mémoire ↔ Compétences

Mémoire pure (produit acheté)	Compétence pure (checklist déploiement)	Hybride (induire compétence depuis expérience réussie)
----------------------------------	--	---



COMMENT ?

Représenter une compétence

4

Section 4

5

Section 5

6

Section 6

7

Section 7

8

Section 8



4

Compétences Rédigées par l'Humain : Standard Agent Skills

Standard Agent Skills, structure, divulgation progressive

4.1 Quand Créer une Compétence ?

- Instructions répétées à l'agent (linters, style code, conventions)
- Spécification fixe, interprétation contextuelle par l'agent

4.2 Structure Standard (Dossier de Compétence)

```
ma_competence/
├── skill.md           "kw">class="c1"># OBLIGATOIRE
├── scripts/          "kw">class="c1"># Code exécutable (optionnel)
├── references/       "kw">class="c1"># Documentation supplémentaire (optionnel)
└── assets/           "kw">class="c1"># Templates, ressources (optionnel)
```

4.3 Format `skill.md`

5

Évaluation des Compétences : SkillsBench

Benchmark 2M compétences, 400 tâches, 18 modèles

Méthodologie

- **2M compétences** collectées en ligne
- **400 tâches** annotées indépendamment
- Annotateurs sélectionnent compétences pertinentes par tâche
- Test sur **4 frameworks × 18 modèles**

Résultats Clés

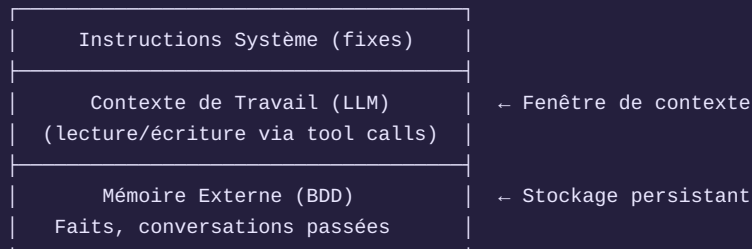
Observation	Implication
Taux de réussite moyen ↑ avec compétences curatées	Les compétences aident globalement
Tâches nécessitant peu de compétences bénéficient plus	Trop de compétences = distraction
Compétences nuisent sur certaines tâches/modèles	Risque d'utilisation inappropriée
Modèles créant compétences <i>de novo</i> → performance ↓	Nécessite expérience réelle comme base

6

Fondations : Mémoire Externe et Réflexion

MEMGPT, Mem0, Reflection, RAG trajectoires

6.1 MEMGPT (2023) — Architecture Mémoire Hiérarchique



- Lectures/écritures contrôlées par tool calls du modèle
- Inspirée systèmes d'exploitation (mémoire rapide/lente)

6.2 Mem0 — Mise à Jour de Mémoire

Opérations sur faits existants :

Cas	Action
Nouveau fait	ADD
Mise à jour (changement emploi)	UPDATE
Négation ("je ne travaille plus chez X")	DELETE
Non pertinent	IGNORE

*Analogie avec ****Delta/Linear Attention**** : mise à jour de valeurs stockées par clé.*

6.3 Reflection — Apprentissage par Retours Naturels

- Échec tâche → Feedback en langage naturel ("Tu n'as donné qu'une bataille, pas la série")
- Conditionner la 2e tentative sur ce feedback
- Généralisable : feedback assez général → aide tâches futures similaires

6.4 Récupération de Trajectoires (RAG pour Agents)

- Pool d'expériences passées (trajectoires + feedback optionnel)
- Nouvelle tâche → Retrieval (embedding description tâche) → Contexte LLM
- Efficace mais : qualité du retriever et du juge critiques

7

Induction Automatique de Compétences

Apprentissage en ligne, AWM (texte), ASI (code)

7.1 Cadre : Apprentissage en Ligne (Online Learning)

```
Tâche 1 → Induction → Compétence A, B
Tâche 2 → Utilise A,B → Induction → Compétence C
Tâche 3 → Utilise A,B,C → ...
      ↓
Taux de réussite croissant ?
Tâches plus complexes résolues ?
```

7.2 Agent Workflow Memory (AWM) — Induction Textuelle

Processus :

1. Trajectoire complète (tool calls) générée
2. LLM identifie sous-tâches réutilisables → **Workflows**
3. Workflow = description + séquence d'actions **templatisée** (variables abstraites)
4. **Juge** évalue si trajectoire correcte → décide stockage

Exemple Workflow :

```
Description: "Rechercher un produit sur le site"
Template:
1. navigate(store_url)
2. click(search_box_selector)
3. type("{query}") ← Variable abstraite
4. click(search_button)
```

7.3 Agent Skill Induction (ASI) — Induction Programmatique

Même principe mais sortie = **fonctions Python** :

```
"kw">class="kw">def search_product(query: str, category: str = "kw">None):
    """Recherche un produit sur le site e-commerce."""
    click(SELECTORS['search_box'])
    type(query)
    "kw">if category:
        select_category(category)
    click(SELECTORS['search_button'])
    "kw">return get_results()
```

- Trajectoire réécrite en appelant ces fonctions
- **Test par exécution** : on lance le code généré + juge sur résultat final
- Si succès → ajout à la bibliothèque d'outils (devenant tool calls)

Critère	Compétences Textuelles (Workflows)	Compétences en Code (Fonctions)
Flexibilité	Modèle adapte les actions au contexte	Rigide : exécution identique
Robustesse changements UI	Modèle peut choisir bon sélecteur	Casse si sélecteurs changent
Composition	Difficile	Naturelle (fonctions appellent fonctions)
Testabilité	Difficile	Exécution + test unitaire possible
Linter/Analyse statique	Non	Oui (détecte params inutilisés, bugs)
Efficacité (tokens)	Re-génère chaînes de pensée + observations	1 tool call = N actions (gain énorme)
Réutilisation cross-site	Meilleure (abstraction haute)	Faible (dépend implémentation concrète)

Résultats Empiriques (ASI)

- Code > Texte > Pas de mémoire sur : **précision** ET **efficacité** (nombre d'interactions env.)
- Mais code fragile sur **généralisation cross-site**

8.1 Solution Hybride : PolySkill (Classes Abstraites)

```
"kw">class="c1"># Classe abstraite (induite 1re fois)
"kw">class ECommerceSite:
    "kw">class="kw">def search_product(self, query): ...
    "kw">class="kw">def add_to_cart(self, product_id): ...
    "kw">class="kw">def checkout(self): ...

"kw">class="c1"># Implémentation site-spécifique
"kw">class AmazonSite(ECommerceSite):
    "kw">class="kw">def search_product(self, query):
        "kw">class="c1"># sélecteurs Amazon
        ...

"kw">class TargetSite(ECommerceSite):
    "kw">class="kw">def search_product(self, query):
        "kw">class="c1"># radio buttons Target, pas dropdown
        ...
```

- Changement de site → nouvelle implémentation, même interface
- Permet grande bibliothèque, sous-ensemble actif selon contexte



COMMENT LE GÉRER ?

Le cycle de vie

9

Section 9

10

Section 10

11

Section 11



9

Cycle de Vie des Compétences

Opérations CRUD, SkillWeaver, boucle raffinage

Opérations de Gestion

Opération	Déclencheur	Méthode
Création	Nouvelle sous-tâche réussie	Induction (AWM/ASI)
Validation	Avant stockage	Juge sur trajectoire / Exécution code
Révision	Échec utilisation / Linter feedback	Reflection + ré-écriture
Suppression	Obsolète, non utilisée, conflit	Politique LRU / Fréquence d'usage
Fusion	Compétences redondantes	Détection similarité + LLM merge
Généralisation	Trop spécifique	Abstraction paramètres / Classe mère

SkillWeaver — Boucle de Raffinage

```

"kw">while "kw">not success:
    skills = propose_skills(task)
    episode = execute_with_skills(task, skills)
    reward = judge(episode)
    "kw">if reward.success:
        store(skills)
    "kw">else:
        feedback = linter(skills) + reward.feedback
        skills = revise(skills, feedback)

```

10

Apprendre des Échecs (Synapse / ReasoningBench)

Synapse, stratégies correctives, qualité juge

Limite de l'Apprentissage par Succès Seul

- Échec = information précieuse sur **ce qu'il ne faut pas faire**
- Ex: Requête trop large → 5000 résultats → pagination infinie → échec

Approche Synapse

1. Trajectoire échouée → LLM génère **stratégie corrective** (texte)

> "Utiliser terme plus spécifique, afficher plus d'items/page"

1. Stocker stratégie (positif + négatif) en mémoire
2. Retrieval + conditionnement sur futures tâches

Résultats Comparatifs

Type Mémoire	Gain avec Exemples Négatifs
Trajectoires complètes	✗ Aucun
Workflows (AWM)	✗ Aucun
Stratégies textuelles (Synapse)	✓ Substantiel

Le texte permet d'exprimer **comment ne pas échouer**** de façon générale, contrairement aux exemples concrets.**

Qualité du Juge (ReasoningBench)

- Juge imparfait = bruit dans la mémoire
- **Zone optimale** : précision juge ~70-90% → encore gains
- En-dessous : performance s'effondre
- Piste : entraîner meilleurs juges / signaux de récompense plus riches

11

Gestion de la Taille de la Mémoire

Distraction, retrieval, cache LRU, fusion

Problème : TROP de Compétences = Distraction

- SkillsBench : performance ↓ quand beaucoup de compétences disponibles
- Raison : contexte plus long, modèles distraits par compétences non pertinentes

Solutions

Approche	Principe
Retrieval sélectif	Embedding + top-k pertinent seulement
Conditionnement domaine	Charger seulement compétences du site/domaine courant
Cache LRF (Least Recently/Frequently Used)	Éviction compétences peu/plus utilisées
Regroupement/Fusion	Dédupliquer, généraliser compétences similaires

Résultats Cache LRU

- Réduction taille contexte → **efficacité** ↑
- Moins distraction → **précision** ↑ (parfois)



IV

ET APRÈS ?

Optimiser & évaluer

12

Section 12

résumé

Section résumé

références

Section références

exercices

Section exercices



12

Apprentissage par Renforcement pour l'Induction

RL bout-en-bout, récompense réutilisation, ACL 2024

Idée Centrale

Au lieu de *prompter* le LLM pour induire/utiliser compétences → **entraîner** par RL sur le processus complet.

Boucle RL (RL4SKILLS, SkillForge)

État: Tâche courante + Mémoire compétences

Action: Induire compétence / Utiliser compétence / Réviser

Récompense:

- Succès tâche courante
- + Bonus si compétence passée réutilisée
- - Pénalité taille bibliothèque (régularisation)

Avantages

- Optimisation bout-en-bout (induction + récupération + utilisation)
- Apprend *quelles* compétences induire, *quand* les utiliser
- Peut intégrer contraintes (taille mémoire, généralisation)

Résultats Récents (ACL 2024)

- RL sur séquences tâches similaires → **amélioration significative**
- Terme de récompense "réutilisation" crucial
- Direction prometteuse : co-entraînement agent + induction compétences